

レポート課題 2

05-191009 奥村真司

問 1

動作例

```
# let five = i2n 5;;
val five : nat = S (S (S (S (S Z))))
# let three = i2n 3;;
val three : nat = S (S (S Z))
# add five three;;
- : nat = S (S (S (S (S (S (S (S Z)))))))
# sub five three;;
- : nat = S (S Z)
# mul five three;;
- : nat = S (S (S (S (S (S (S (S (S (S (S (S (S Z))))))))))))))
# n2i (pow five three);;
- : int = 125
# n2i (mul five three);;
- : int = 15
```

考察

まず nat 型はパターンマッチによって Z か S の判定をし、S ならば 1 引いたものがとれる (すなわち `|S p -> ...`) ことに注意する。add,sub,mul,pow は第 2 引数をパターンマッチで場合分けし、一つ小さい場合に帰着して再帰した。他も同様に Z になるまで一つずつ小さいものに帰着して再帰した。

問 2

動作例

```
# let t = Node(1,Node(2,Node(3,Leaf,Leaf),Node(4,Leaf,Leaf)),
Node(5,Node(6,Leaf,Leaf),Node(7,Leaf,Leaf)));;
val t : int tree =
  Node (1, Node (2, Node (3, Leaf, Leaf), Node (4, Leaf, Leaf)),
    Node (5, Node (6, Leaf, Leaf), Node (7, Leaf, Leaf)))
# dfs_pre t;;
- : int list = [1; 2; 3; 4; 5; 6; 7]
# dfs_in t;;
- : int list = [3; 2; 4; 1; 6; 5; 7]
# dfs_pos t;;
```

```
- : int list = [3; 4; 2; 6; 7; 5; 1]
```

考察

今いる頂点を n , 左の部分木を l , 右の部分木を r とする。行きがけ順はまず n をマークしてから次に l の中を行きがけ順でマークして、次に r の中を行きがけ順でマークする順であり、通りがけ順は l, n, r の順、帰りがけ順は l, r, n の順である。二分木の型の定義よりこれら n, l, r はパターンマッチで簡単に得られるので各々の順の決まりをそのまま再帰で実装してやるとそれぞれの順で並べたリストが得られる。

問 3

動作例

```
# let t = Node(1, Node(2, Node(3, Leaf, Leaf), Node(4, Leaf, Leaf)),
Node(5, Node(6, Leaf, Leaf), Node(7, Leaf, Leaf)));;
val t : int tree =
  Node (1, Node (2, Node (3, Leaf, Leaf), Node (4, Leaf, Leaf)),
    Node (5, Node (6, Leaf, Leaf), Node (7, Leaf, Leaf)))
# bfs t;;
- : int list = [1; 2; 5; 3; 4; 6; 7]
```

考察

幅優先探索の順に出力したいので `queue` があれば良い。 `list` を後ろへの挿入に時間のかかる `queue` と思って、BFS をしてリストを作った。現在の `queue` と作成途中のリストを引数にとって末尾再帰で実装した。

問 4

定義

```
type expr =
| EConstInt of int
| EAdd of expr * expr
| ESub of expr * expr
| EMul of expr * expr
| EConstBool of bool
| Eequal of expr * expr
| Elt of expr * expr
| Eif of expr * expr * expr;;
type value = VInt of int | VBool of bool;;
```

考察

BNF の通り定義するだけ。

問 5

動作例

```
# eval (EAdd (EConstInt 5,(EMul (EConstInt 2,EConstInt 3))));;
- : value = VInt 11
# eval (EAdd (EConstInt 5,(EMul (EConstInt 2,EConstBool false))));;
Exception: Eval_error.
# eval (Eif ((Eequal (EConstInt 5,EAdd (EConstInt 2,EConstInt 3))),
EConstInt 4,EConstBool true));;
- : value = VInt 4
# eval (Eif ((Eequal (EConstInt 5,EAdd (EConstInt 2,EConstInt 2))),
EConstInt 4,EConstBool true));;
- : value = VBool true
# eval (EDiv (EConstInt 5,ESub (EConstInt 3,EConstInt 1))));;
- : value = VInt 2
# eval (EDiv (EConstInt 5,ESub (EConstInt 3,EConstInt 2))));;
- : value = VInt 5
# eval (EDiv (EConstInt 5,ESub (EConstInt 3,EConstInt 3))));;
Exception: Eval_error.
# eval (Eif ((Eequal (EConstInt 5,EAdd (EConstInt 2,EConstInt 3))),
EConstInt 4,EAdd (EConstInt 3,EConstBool true))));;
- : value = VInt 4
```

考察

基本的にはパターンマッチで型がおかしくないかを検査する。EAdd 等の四則演算では両引数の評価結果が VInt かどうかをチェックしてから演算をする。Elt は bool 同士の演算もできる仕様とした。(true のほうが false より大きい)

また、Eif においては第一引数の評価結果によって一方のみが評価されるようにしたので評価されない方の式が不正 (bool と int の足し算など) でも、エラーを返さないという仕様にした。(上の動作例で最後の例)

発展 1

動作例

```
# let sum_to_sub = fun f n -> if n=0 then 0 else n + f (n-1);;
val sum_to_sub : (int -> int) -> int -> int = <fun>
# let sum_to = fun x -> fix sum_to_sub x;;
val sum_to : int -> int = <fun>
# sum_to 5;;
- : int = 15
# sum_to 10;;
- : int = 55
```

考察

`let rec` を用いずに再帰関数を定義するために問 2 の二分木などで扱った再帰的な型の定義に着目し、それを利用した。advanced1.ml 内で定義されている `recfunc` 型は、それ自身を引数として [何か] を返す関数である。この定義によって `recfunc` 型の関数は自分自身を引数としてとることができる。`rec_call` でやっていることがこれである。

今回最終的に `fix` を作りたいので [何か] は `fix` と同じ型の関数とした。`fix_rec` は、`recfunc` 型で `rec_call fix_rec` が `fix` と同等の関数を返すようなものである。定義を見るとわかるように、`fix_rec` は自分自身 (`self`) を引数にとると、返すときに自分に自分を食わせる (`rec_call self`) ようになっておりこれが再帰的に起こることで、`fix` の再帰が実現されている。`fix` は第 1 回でやったとおり `fix f x = f (fix f) x` をみたすものだったのでこれと合致するように `fix_rec` を定義した。

発展 2

動作例

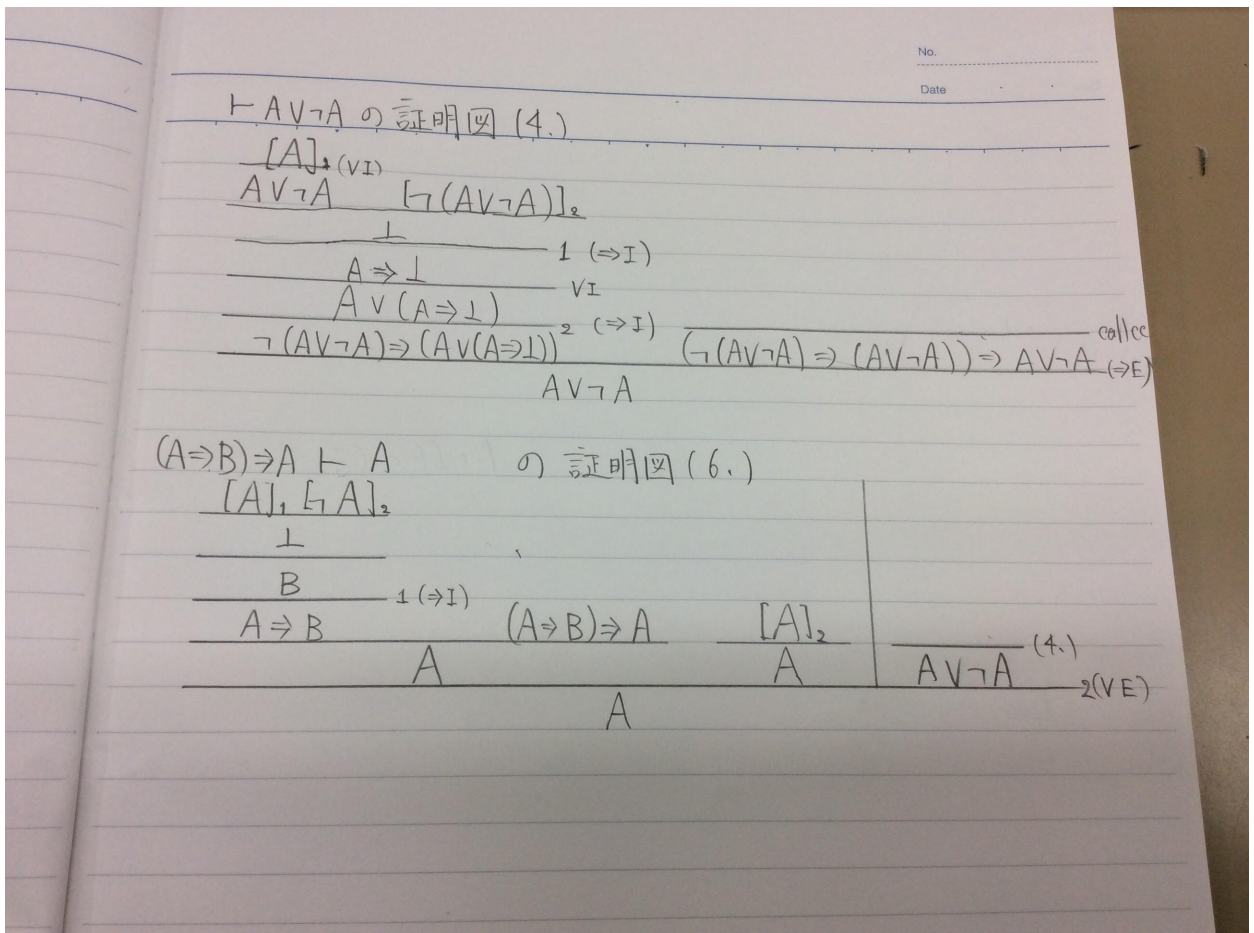
```
# #use "advanced2.ml";;
type false_t = { t : 'a. 'a; }
type 'a not_t = 'a -> false_t
type ('a, 'b) and_t = 'a * 'b
type ('a, 'b) or_t = L of 'a | R of 'b
val top : 'a -> 'a = <fun>
val orE : ('a, 'b) or_t -> ('a -> 'c) -> ('b -> 'c) -> 'c = <fun>
val func1 : ('a -> 'b) -> ('b -> 'c) -> 'a -> 'c = <fun>
val func2 : ('a, ('b, 'c) and_t) or_t -> (('a, 'b) or_t, ('a, 'c) or_t) and_t =
  <fun>
val func3 : (('a, 'b) or_t, ('a, 'c) or_t) and_t -> ('a, ('b, 'c) and_t) or_t =
  <fun>
val callcc : (('a -> false_t) -> 'a) -> 'a = <fun>
val func4 : ('a -> 'c) -> ('a not_t -> 'c) -> 'c = <fun>
val func6 : (('a -> 'b) -> 'a) -> 'a = <fun>
```

考察

カーリーワード対応より直観主義論理で示することができる命題に対応する型ならば再帰、副作用なしで定義可能で、実際 1,2,3 は定義できた。(単純なので証明図は省略する)

4 は排中律だが、直観主義論理では示せないので再帰、副作用なしでは定義できない。5 も直観主義論理において偽の命題なので定義できない。6 の Peirce 則も同様に定義できない。

次に `callcc` を許す場合を考える。`callcc` を許すと古典論理で証明可能な命題に対応する型のものなら定義可能になる。よって 4,6 が定義できる。5 は古典論理においても偽の命題なので定義できない。`callcc` の型はスライドにあるとおり $(\text{not } A \rightarrow A) \rightarrow A$ である。直観主義論理の推論規則に `callcc` を公理として加えた体系での 4,6 の証明図を以下に載せる。(打ち込むのが面倒で手書きです、すみません) 4 の証明図の最後のステップでは `not A` が `A -> bottom` であることを用いています。この証明図の流れに沿って順に組み立てていくことで advanced2.ml 内の `func4`, `func6` が定義できた。今回 `callcc` を型だけ合うように定義して代用しているため 4 でスライドの上の式を



作ろうとするとこれは値なので評価が始まって代入している callcc の実装のせいで無限ループしてしまうため下の式の型の関数を定義した。